Smart Ambulance IoT

Internet of Things

Giuseppe Di Blasi
Università di Parma

Two CoAP Servers, Three CoAP Clients

Servers

- `Cockpit` → port 5684
- `RearCabin` → port 5683
- Both host the same three resources: `TemperatureSensor`, `AirConditioningController`, `HMI`

Clients

- `Driver`, `Reliever`, `GPS`
- All extend a common abstract class `SmartAmbulanceClient`
- Shared reusable GET/PUT operations toward HMI, sensor and controller

Design choice

Two independent `CoapServer` instances instead of one server with parametrized resources — keeps the two environments fully separated.

Message Format: Hand-Built JSON

- Payloads built with `String.format`
- Parsed with a small helper: `extractJsonValue(json, key)`
- Every payload carries a `"Client"` field identifying the sender

```
{
  "Client": "DRIVER",
  "temp_target": 21
}
```

```
extractJsonValue(
  payload, "temp_target")
```

Why it matters

The `Client` field is what the server uses to route authorization logic (next slide).

Authorization Rules on the AC Controller

Driver

- Cockpit → allowed
- Rear Cabin → allowed

Reliever

- Cockpit → FORBIDDEN
- Rear Cabin → allowed

```
if (client.equals("RELIEVER") && port == COCKPIT_PORT) {
    exchange.respond(ResponseCode.FORBIDDEN);
    return;
}
```

Note

The check is done inside `handlePUT`, comparing the `Client` field with the server's own port.

Dynamic Temperature + Observe Pattern

Gradual convergence

- `TimerTask` with `scheduleAtFixedRate`
- Current temperature moves $\pm 1^{\circ}$C every 1.5s toward the target
- Triggered by `setTargetTemperature()`

CoAP Observe

- A client can observe the temperature sensor
- The subscribed client is updated at every change
- Unsubscribes with `stopObservingTemperature()`

Design choice

Simulating a realistic air-conditioning response instead of an instantaneous jump to the target temperature.

HMI Synchronization Between Environments

- When Driver or Reliever update emergency level / location, the `HMI` resource itself opens an internal `CoapClient`
- Same thing applies for the GPS when it sends new coordinates
- It sends a PUT to the twin HMI on the other server — a **server-to-server** propagation, not client-to-client

```
updateOtherHMI(): PUT to twin port, payload marked "HMI_UP"
```

End-to-End Simulation

`SmartAmbulanceSimulation.main` orchestrates a scripted scenario, with `Thread.sleep` pauses between steps:

1. Driver sets the temperature in Rear Cabin
2. GPS sends coordinates to the HMI
3. Driver updates emergency/location (propagated to the other HMI)
4. Reliever attempts the same PUT in Cockpit $\rightarrow$ FORBIDDEN
5. Driver switches server and repeats GET/PUT in Cockpit
6. Reliever observes the temperature sensor, then cancels

Questions & Contacts

Any questions?

E-mail: giuseppe.diblasi@studenti.unipr.it